Разбор Bulls and Cows: TDD & Функциональное программирование

Чистые функции, каррирование, мемоизация и рекурсия



Gemini Notebook

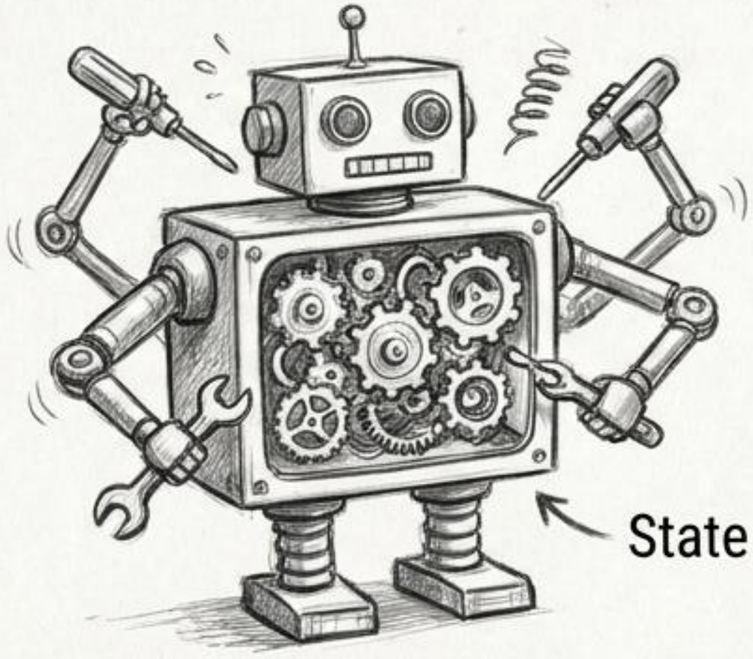
Давайте знакомиться!



Привет! Я Маргарита Малец, Software Engineer в EPAM. Я окончила RS School в июле 2021 года, а уже в марте 2022-го впервые стала ментором. С тех пор школа стала неотъемлемой частью моей жизни. Я уверена, что постоянно расту и учусь вместе со своими менти.
Моё любимое занятие — проводить технические интервью.

Gemini Notebook

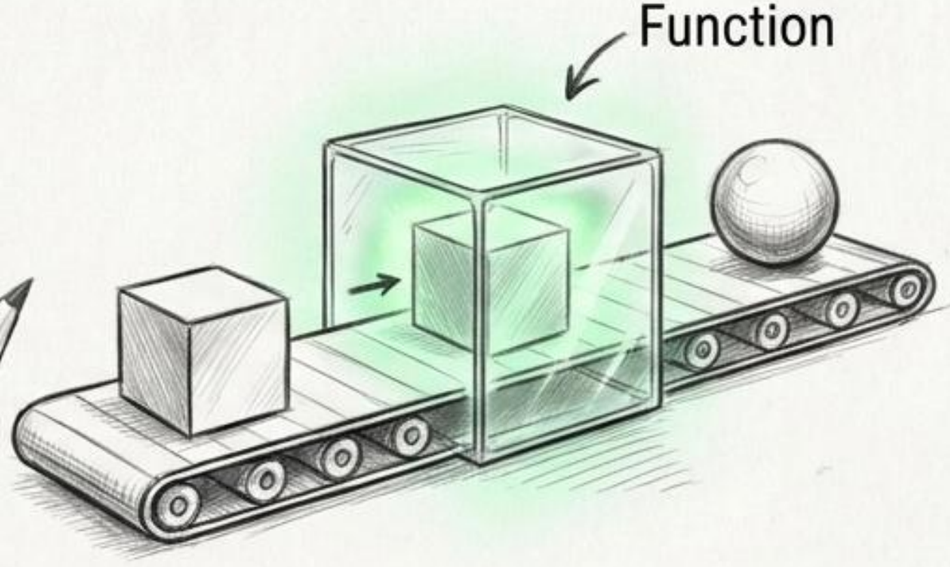
ФП vs ООП — Как мы мыслим?



Объект несет в себе состояние
и сам его меняет

```
class Counter {
    count = 0;
    increment() { this.count++; }
}
```





Функции не меняют мир —
они возвращают новые значения

```
const increment = (n: number): number => n + 1;
```

Gemini Notebook

Фундамент: Чистые функции и Иммутабельность






```
// Плохо (Мутация)
history.push(guess);
return history;
```



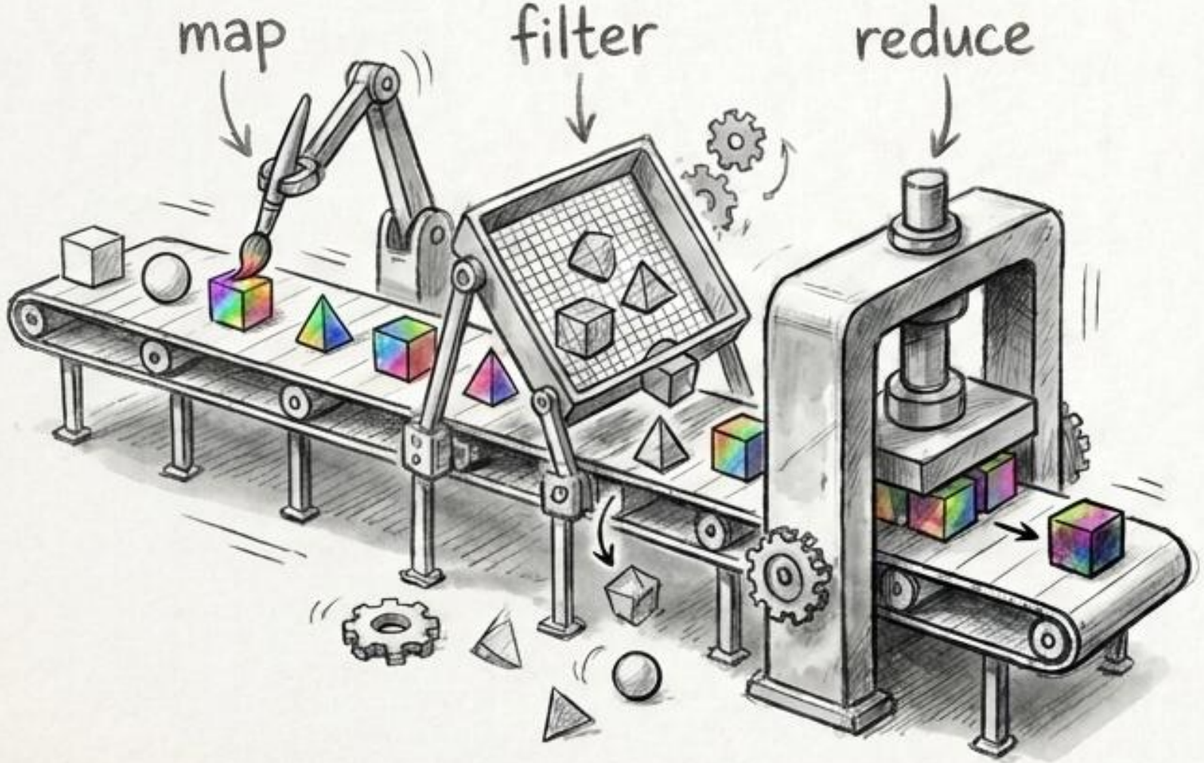


```
// Хорошо (Новые данные)
return [...history, guess];
```

Gemini Notebook

Фабрики функций: HOF и Каррирование

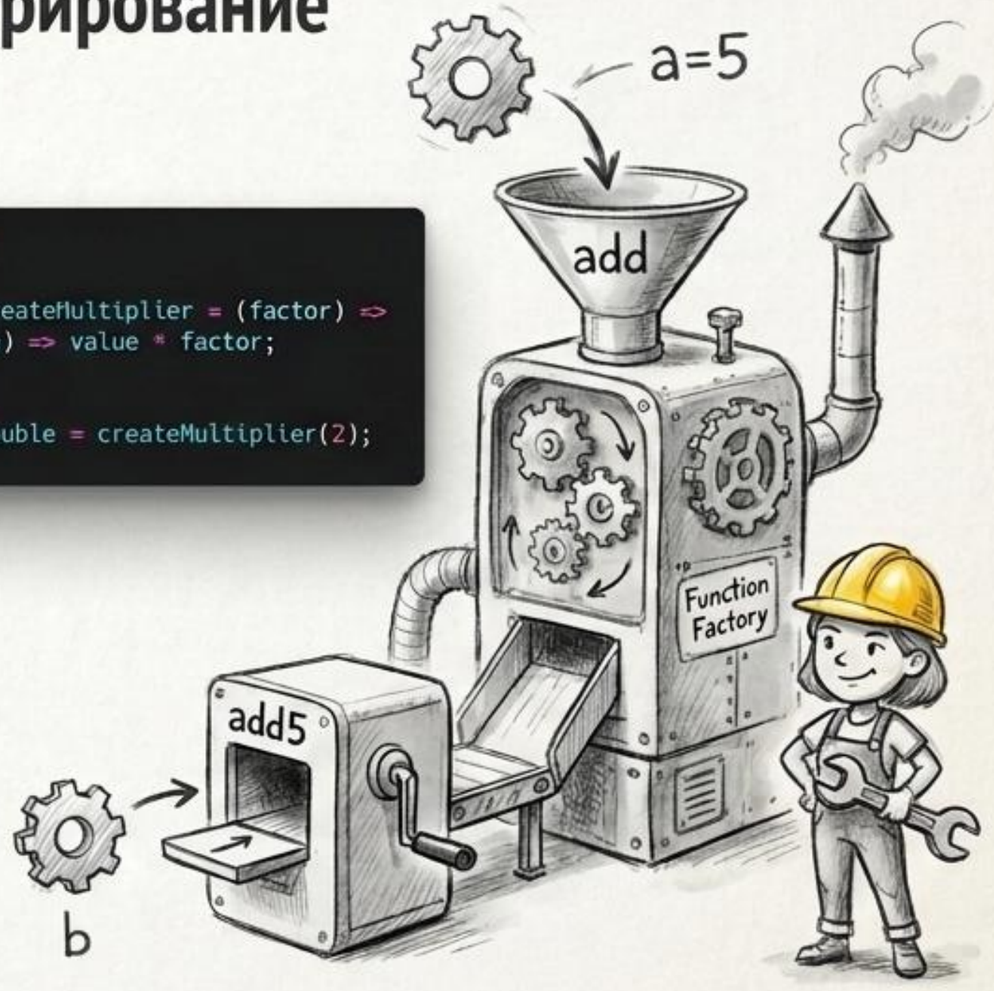
HOF - High Order Function (Функции высшего порядка)



Каррирование

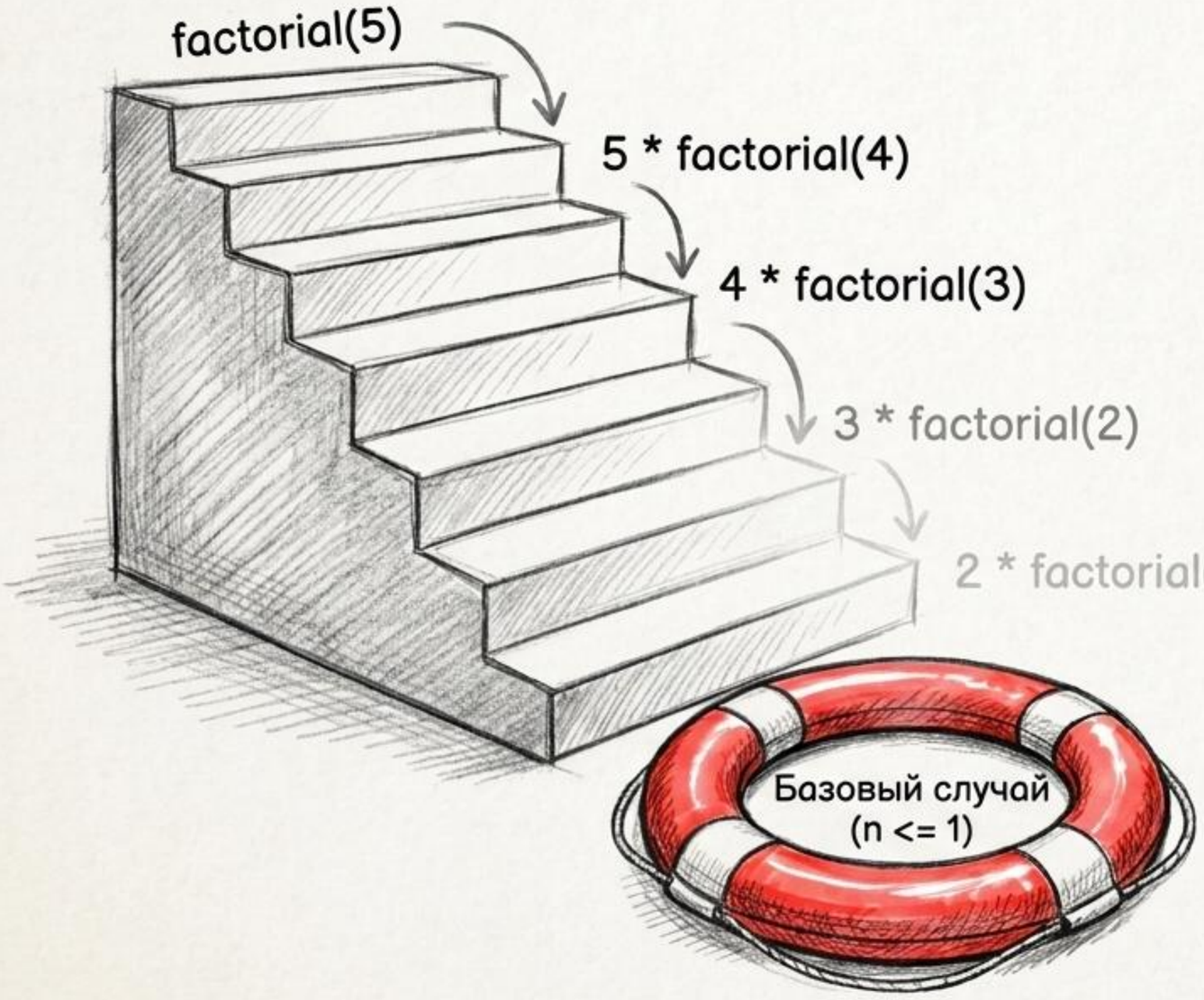
```
const createMultiplier = (factor) =>
  (value) => value * factor;

const double = createMultiplier(2);
```

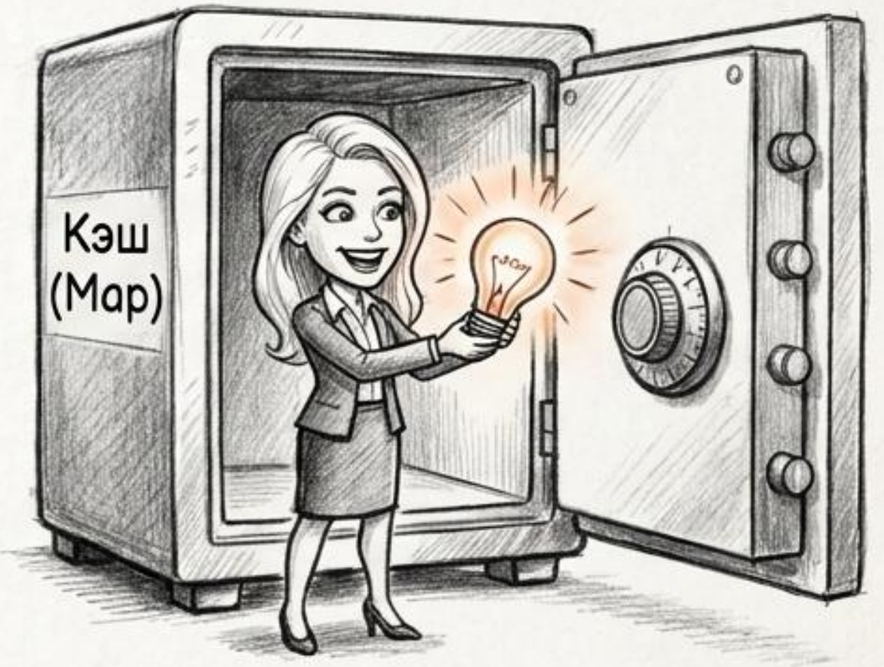


Gemini Notebook

Умные циклы: Рекурсия и Мемоизация



Второй вызов возвращает
ответ без вычислений




```
const key = JSON.stringify(args);
if (cache.has(key)) return cache.get(key)!;
// ...вычисление и cache.set()
```

Gemini Notebook

Время практики! Что выведет консоль?

```
const result = [1, 2, 3, 4, 5]
  .filter(n => n % 2 !== 0)
  .map(n => n * n)
  .reduce((acc, n) => acc + n, 0)
console.log(result);
```



А) 15

Б) 35

В) 55

Г) Ошибка: мутация массива

Gemini Notebook

И правильный ответ... Б) 35!

А) 15

Б) 35

В) 55

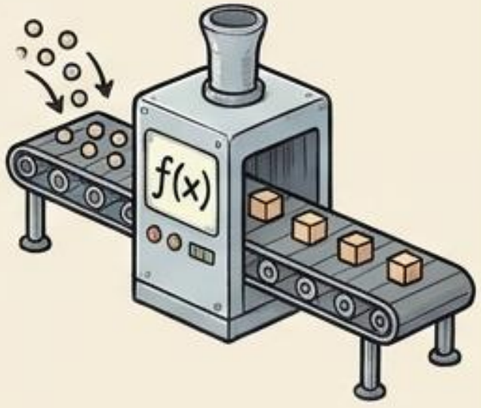
Решение:

[1, 2, 3, 4, 5] -> filter(n => n % 2 !== 0)
-> [1, 3, 5] -> map(n => n * n) -> [1, 9, 25]
-> reduce((acc, n) => acc + n, 0) -> 35



Gemini Notebook

Кто тут лишний?



А) Map



Б) Filter



В) Reduce



Г) For

Gemini Notebook

Правильный ответ Г) for (let i...) {} - Монстр мутации!



Цикл for с ручными счетчиками и мутацией:

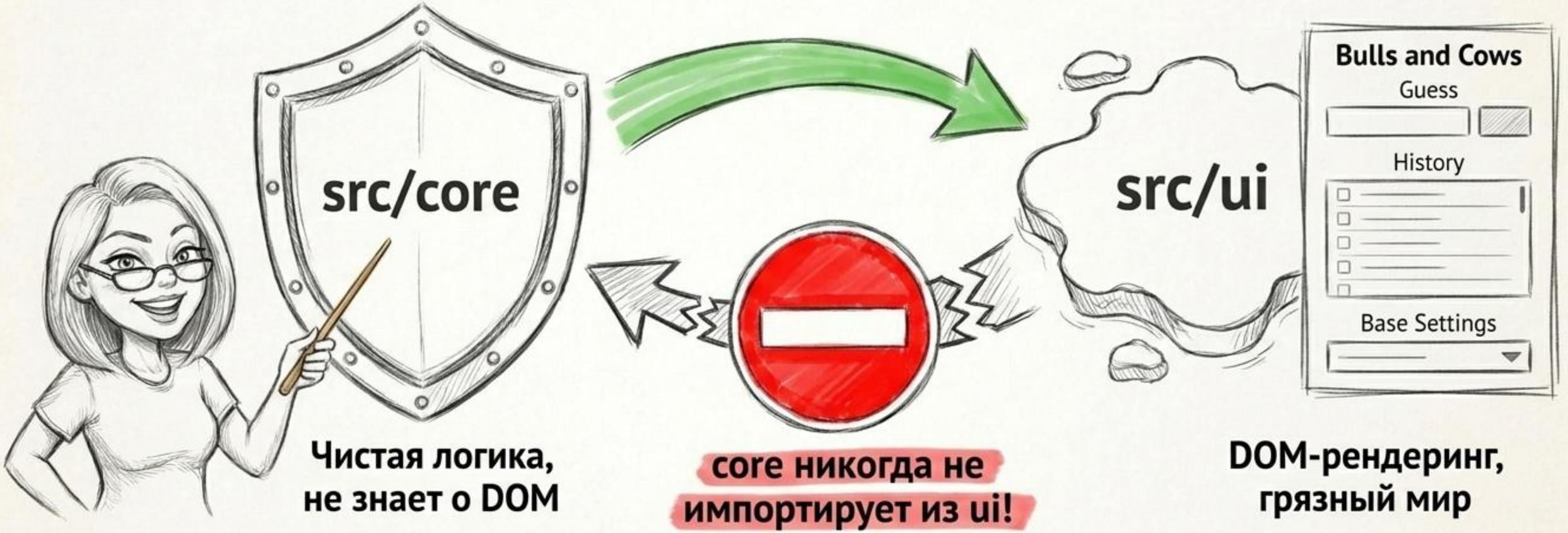
```
for (let i = 0; i < arr.length; i++) {
    arr[i] = i * 2;
}
```



Это декларативный подход - чистые HOF – они возвращают новые значения и не меняют оригинал.

Gemini Notebook

План игры: Архитектура и UI



Чистая логика, не знает о DOM

core никогда не импортирует из ui!

DOM-рендеринг, грязный мир

Почему TDD? Потому что тестировать чистые функции ядра (core) можно в Node.js, быстро и без браузера!

Gemini Notebook

Подготовка песочницы (Terminal)

```
(Инициализация)
npm init -y
npx tsc --init
```

```
(Инструменты)
npm install --save-dev typescript @types/node
npm install --save-dev vitest
npm install --save-dev eslint prettier husky lint-staged
```

```
(Настройка Husky)
npx husky init
echo 'npm test' > .husky/pre-push
git commit -m "init: install dependencies" && git pu
```



❗ **Обратите внимание на “module”: “NodeNext” в tsconfig.**
В импортах нужно писать .js, даже если файл .ts!

Gemini Notebook

Шаг 1: Каррирование в TDD (createDigitValidator)

```
// src/core/createDigitValidator.test.ts

const isHex = createDigitValidator(16);

expect(isHex('a')).toBe(true);
expect(isHex('g')).toBe(false); // Красный, падает
```



```
// src/core/createDigitValidator.ts
const ALL_DIGITS = '0123456789abcdef';

export function createDigitValidator(base: number) {
  const validDigits = ALL_DIGITS.slice(0, base);
  // Каррирование: возвращаем функцию!
  return (char: string) => validDigits.includes(char.toLowerCase());
}
```

```
$ git commit -m "feat: add failing tests for createDigitValidator"
$ git commit -m "feat: implement createDigitValidator"
$ git push
```

Один раз передали систему счисления (base), возвращенная функция её «помнит».

Gemini Notebook

Шаг 2: HOF в бою (evaluateGuess)

Нет счетчика i++!

```
// Считаем Быков (совпадение позиции) через reduce
const bulls = s.reduce((count, char, i) =>
   count + (char === g[i] ? 1 : 0), 0);

// Считаем Коров (пересечение множеств)
// countChars - тоже reduce!
const common = Object.keys(secretCounts).reduce((total, c) =>
   total + Math.min(secretCounts[c], guessCounts[c] ?? 0), 0);
```

Считаем частоту символов и берем минимум



Gemini Notebook

Теория стала Практикой



Матрица Синтеза	
Концепты	В проекте Bulls and Cows
1. Каррирование	1. createDigitValidator(base)
2. Функции высшего порядка (HOF)	2. evaluateGuess (без циклов for)
3. Рекурсия и Мемоизация	3. countPossibleSecrets (Map-кэш)
4. Иммутабельность	4. [...history, entry]

Gemini Notebook

Спасибо за внимание! Готова ответить на вопросы.

Вопросы и Ответы (Q&A)



- Задание Bulls and Cows
- Основы ФП (RS School)
- MDN: First-class function



Не забудьте про DESIGN.md (20 баллов)!

Gemini Notebook